

Projet Python 1SIO janvier 2025

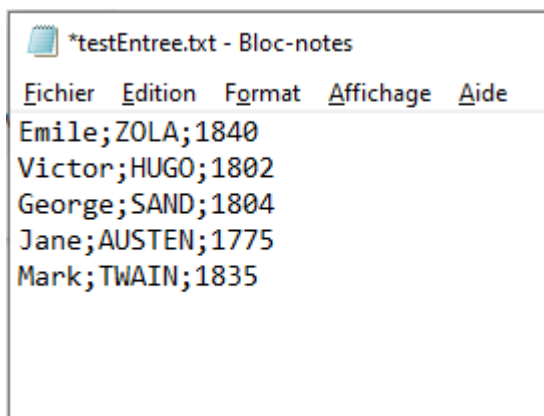
Contexte :

La société STESIO utilise un proxy pour enregistrer les accès au web réalisés par ses salariés, conformément aux obligations légales. Ces fichiers journaux contiennent des informations sensibles (adresses IP, dates, heures, et URL consultées), et leur volume important nécessite un traitement structuré. Pour cela, une base de données nommée logs_web sera créée pour stocker et structurer ces informations.

Objectifs :

- Structurer les données issues des fichiers journaux dans une base de données relationnelle.
- Faciliter l'analyse des logs pour produire des statistiques, telles que :
 - o Les sites les plus visités.
 - o Les utilisateurs consommant le plus de ressources.
- Répondre à des besoins spécifiques, comme :
 - o Identifier les utilisateurs ayant accédé à un site donné à une date et une heure précise.

Etape 1 :



Création du fichier testEntree.txt

```

1  import random
2  def triPersonnalite(nom_du_fichier) :
3      with open(nom_du_fichier, 'r') as f:
4          contenu = [ligne.strip().split(";") for ligne in f]
5      return contenu

```

Cette fonction permet d'ouvrir le fichier et de rentrer dans une liste les éléments lignes par lignes.

```

9  def calculAge(nom):
10     nbrElement=len(testEntree)
11     for i in range(nbrElement):
12         testEntree[i][0]
13         if nom==testEntree[i][0]:
14             k=testEntree[i][2]
15             return 2025-int((k))
16     return "Personne nom trouvée"
17
18 ageChercher=str(input("De quelle personne cherchez-vous l'âge ?"))
19 print(calculAge(ageChercher))

```

Cette fonction permet de calculer l'âge à partir du prénom de la personne.

```

21 def generateurPseudo(nomP):
22     for personne in testEntree:
23         if nomP == personne[0]:
24             prenom = personne[0]
25             nom = personne[1]
26             annee = personne[2]
27             ch1 = prenom[0]
28             ch2 = nom[0]
29             ch3 = nom[1] if len(nom) > 1 else 'x'
30             ch4 = nom[2] if len(nom) > 2 else 'x'
31             ch5 = str(calculAge(nomP))
32             ch6 = str(random.randint(10, 99))
33             pseudo = (ch1 + ch2 + ch3 + ch4 + ch5 + ch6).lower()
34             return pseudo
35     return "Personne non trouvée"
36
37 pseudoChercher=str(input("De quelle personne voulez-vous avoir le pseudo ?"))
38 print(generateurPseudo(pseudoChercher))

```

Cette fonction permet de générer un pseudo à partir du prénom de la personne en prenant la première lettre du prénom, les trois premières lettres du nom, l'âge de la personne et un nombre aléatoire entre 10 et 99.

```

40 def ecriturePseudo(nomP):
41     with open("testSortie.txt", 'w') as g:
42         g.write(generateurPseudo(nomP))
43         g.write("\n")
44
45     ecritureRechercher=str(input("De quelle personne voulez-vous écrire le pseudo ?"))
46     ecriturePseudo(ecritureRechercher)

```

Cette fonction permet de créer un dossier “testSortie.txt” et d’écrire les pseudos de la personne choisit.

Etape 2 :

On renomme la machine sous Linux.

Nom de l'appareil

GALLAISC24 >

On change le network adapter SIO PEDAGO en PlotJ-517 (le bon VLAN)

> Network adapter 1

Plot-J-517 ▾

☒ Connected

Réattribution de l’adresse de la machine.

Méthode IPv4

☐ Automatique (DHCP)
 ☒ Manuel
 ☐ Partagée avec d'autres ordinateurs

Adresses

Adresse	Masque de réseau	
10.15.117.131	255.255.255.0	

```

1 CREATE DATABASE logs_web CHARACTER SET utf8 COLLATE utf8_unicode_ci;

```

Création de la base de donnée logs_web.

✓ MySQL a retourné un résultat vide (c'est à dire aucune ligne). (traitement en 0.0019 seconde(s).)

```
CREATE USER 'utilog' IDENTIFIED BY 'utilog';
```

[[Éditer en ligne](#)] [[Éditer](#)] [[Créer le code source PHP](#)]

Création de l'utilisateur utilog.

✓ MySQL a retourné un résultat vide (c'est à dire aucune ligne). (traitement en 0.0009 seconde(s).)

```
CREATE USER 'gestionlog' IDENTIFIED BY 'gestionlog';
```

[[Éditer en ligne](#)] [[Éditer](#)] [[Créer le code source PHP](#)]

Création de l'utilisateur gestionlog

✓ MySQL a retourné un résultat vide (c'est à dire aucune ligne). (traitement en 0.0011 seconde(s).)

```
GRANT SELECT ON logs_web.* TO 'utilog';
```

[[Éditer en ligne](#)] [[Éditer](#)] [[Créer le code source PHP](#)]

On donne le droit de SELECT à l'utilisateur utilog

✓ MySQL a retourné un résultat vide (c'est à dire aucune ligne). (traitement en 0.0010 seconde(s).)

```
GRANT INSERT, UPDATE, DELETE, SELECT ON logs_web.* TO 'gestionlog';
```

[[Éditer en ligne](#)] [[Éditer](#)] [[Créer le code source PHP](#)]

On donne les droits INSERT, UPDATE, DELETE, SELECT à l'utilisateur gestionlog

✓ MySQL a retourné un résultat vide (c'est à dire aucune ligne). (traitement en 0.0091 seconde(s).)

```
CREATE TABLE employes ( id_employe INT PRIMARY KEY AUTO_INCREMENT , nom VARCHAR(50) not null, prenom VARCHAR(50)not null, email VARCHAR(100)UNIQUE, adresse_ip VARCHAR(15)UNIQUE, date_creation DATETIME DEFAULT CURRENT_TIMESTAMP, date_motif DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP );
```

[[Éditer en ligne](#)] [[Éditer](#)] [[Créer le code source PHP](#)]

Création de la table employes avec les différents types et contraintes demander. Puis on ajoute les données.

✓ 4 lignes insérées. (traitement en 0.0016 seconde(s).)

```
INSERT INTO employes (id_employe,nom,prenom,email,adresse_ip,date_creation,date_motif) VALUES
(1,'DUFONT','Marie','marie.dufont@gmail.com','192.168.2.2','2024-01-01 10:00:00','2024-01-01 10:00:00'),
(2,'DESBOIS','Paul','paul.desbois@gmail.com','192.168.2.1','2024-01-01 10:05:00','2024-01-01 10:05:00'),
(3,'DURATION','Quentin','quentin.duration@gmail.com','192.168.2.3','2024-01-01 10:10:00','2024-01-01
10:10:00'), (4,'LEJAUNE','Laurence','laurence.lejaune@gmail.com','192.168.2.100','2024-01-01
10:15:00','2024-01-01 10:15:00');
```

[Éditer en ligne] [Éditer] [Créer le code source PHP]

✓ MySQL a retourné un résultat vide (c'est à dire aucune ligne). (traitement en 0.0084 seconde(s).)

```
CREATE TABLE journaux_acces ( id_acces INT PRIMARY KEY AUTO_INCREMENT, adresse_ip_employe VARCHAR(15) , horodatage DATETIME
not null, url_consultee TEXT not null, methode_http VARCHAR(10) not null, code_reponse INT DEFAULT null, date_creation
DATETIME DEFAULT CURRENT_TIMESTAMP, date_modification DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
CONSTRAINT fk_adresse_ip FOREIGN KEY(adresse_ip_employe) REFERENCES employes(adresse_ip) );
```

[Éditer en ligne] [Éditer] [Créer le code source PHP]

Création de la table journaux acces avec les différents types et contraintes demander.
Puis on ajoute les données.

✓ 2 lignes insérées. (traitement en 0.0023 seconde(s).)

```
INSERT INTO
journaux_acces(id_acces,adresse_ip_employe,horodatage,url_consultee,methode_http,code_reponse,
date_creation,date_modification) VALUES (1,'192.168.2.2','2024-01-02
10:00:00','http://exemple.com/page1','GET','200','2024-01-01 10:00:00','2024-01-01 10:00:00'),
(2,'192.168.2.1','2024-01-02 10:05:00','http://exemple.com/page2','POST','201','2024-01-01
10:05:00','2024-01-01 10:05:00');
```

[Éditer en ligne] [Éditer] [Créer le code source PHP]

Erreur

Requête SQL : [Copier](#)

```
CREATE TABLE pause_cafe(
    date_creation DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

MySQL a répondu :

#1142 - La commande 'CREATE' est interdite à l'utilisateur: 'gestionlog'@'localhost' sur la table 'pause_cafe'

En se connectant au compte gestionlog en local, on s'aperçoit qu'il n'a pas les droits pour créer une table comme convenue.

```
✓ MySQL a retourné un résultat vide (c'est à dire aucune ligne). (traitement en 0.0003 seconde(s).)

SELECT* FROM employes;

[ Éditer en ligne ] [ Éditer ] [ Créer le code source PHP ]

id_employe nom prenom email adresse_ip date_creation date_motif
```

Néanmoins il peut tout de même rédiger la commande SELECT car le droit lui a été accordé.

```
#default_storage_engine=InnoDB
#innodb_autoinc_lock_mode=2
#
# Allow server to accept connections on all interfaces.
#
#bind-address=0.0.0.0
bind-address=@IP
#
# Optional setting
#wsrep_slave_threads=1
#innodb_flush_log_at_trx_commit=0
```

bind-address=@IP Le fichier mariadb-server.cnf du serveur mariadb pour remplacer BIND-ADDRESS= 127.0.0.1 par BIND-ADDRESS=@IP

```
✓ MySQL a retourné un résultat vide (c'est à dire aucune ligne). (traitement en 0.0003 seconde(s).)

SELECT* FROM employes;

[ Éditer en ligne ] [ Éditer ] [ Créer le code source PHP ]

id_employe nom prenom email adresse_ip date_creation date_motif
```

A distance.

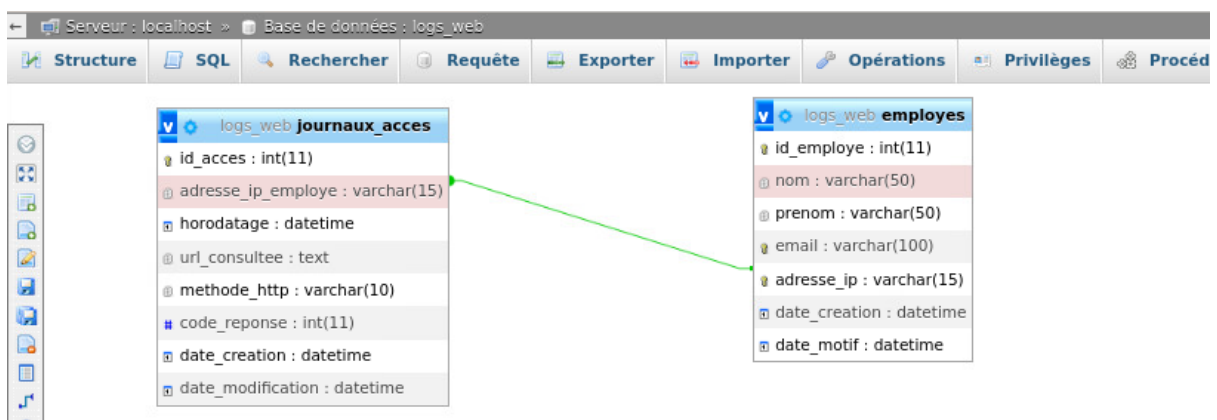


Schéma relationnel de la base de données généré avec le concepteur phpMyAdmin.

Etape 3 :

Explication de l'url de spotify : 00:00:17 192.168.2.2 POST 200 https://www.spotify.com 388+1821 OK

00:00:17 représente l'horodatage, soit la durée de traitement de la requête (en secondes et millisecondes).

192.168.2.2 représente l'adresse ip de l'émetteur de la requête.

POST représente la méthode http utilisé pour envoyer les données jusqu'au server.

200 représente le code http du server, qui ici signal un succès de la requête.

<https://www.spotify.com> Représente l'url du site Spotify

388+1821 représente les données reçues et envoyés en octets

OK signifie que l'opération s'est effectuée avec succès.

Etape 4 :

```
SELECT adresse_ip_employee, url_consultee, COUNT(*) AS visit_count
```

```
FROM log_table
```

```
GROUP BY adresse_ip_employee, url_consultee
```

```
ORDER BY adresse_ip_employee, visit_count DESC
```

Description détaillée du code :

SELECT adresse_ip_employee, url_consultee, COUNT(*) AS visit_count

adresse_ip_employee : Sélectionne l'adresse IP du client dans la table log_table.

Url_consultee : Sélectionne l'URL visitée dans la table.

COUNT(*) : Compte le nombre total d'enregistrements pour chaque combinaison unique de client_ip et url.

AS visit_count : Donne un alias au résultat du comptage pour qu'il soit présenté sous la colonne visit_count.

FROM log_table

- Indique la table source (log_table) à partir de laquelle les données doivent être extraites.

GROUP BY adresse_ip_employee, url_consultee

- Regroupe les données par combinaison unique de client_ip et url.
- Cela permet de calculer le nombre de visites (via COUNT(*)) pour chaque combinaison unique.

ORDER BY adresse_ip_employee, visit_count DESC

- Trie les résultats d'abord par adresse IP du client (client_ip) par ordre croissant (par défaut).
- Ensuite, pour chaque adresse IP, trie les résultats par le nombre de visites (visit_count) dans l'ordre décroissant (DESC).

Etape 5 :

```
1  numFichier = str(input("Entrez le nom du fichier à analyser (23/24/25)"))
2
3  with open("log_proxy_2024-11-" + numFichier + ".txt", 'r') as f:
4      contenu = [ligne.strip().split(" ") for ligne in f]
5
6  nbr = len(contenu)
7  print(contenu[0][1])
8
9  # Une liste pour stocker les utilisateurs uniques
10 unique_ip = []
11 for i in range(nbr):
12     utilisateur = contenu[i][1]
13     if utilisateur not in unique_ip:
14         unique_ip.append(utilisateur)
15
16 print(f"Utilisateurs uniques : {unique_ip}")
17
18 # Une liste pour stocker les visites des utilisateurs et les URLs [utilisateur, url, nombre de visites]
19 visitesParUtilisateur = []
20
21 # Parcourir toutes les lignes pour remplir la liste des visites
22 for i in range(nbr):
23     utilisateur = contenu[i][1]
24     url = contenu[i][4]
25
26     # Vérifier si l'utilisateur est déjà dans la liste des visites
27     etat = False
28     for entre in visitesParUtilisateur:
29         if entre[0] == utilisateur and entre[1] == url:
30             entre[2] += 1 # Compteur
31             etat = True
32             break
33
34     # Si l'utilisateur et l'URL ne sont pas encore dans la liste, les ajouter
35     if not etat:
36         visitesParUtilisateur.append([utilisateur, url, 1])
37
38 # Trouver l'URL la plus visitée pour chaque utilisateur
39 for utilisateur in unique_ip:
40     url_max = None
41     max_visites = 0
42     for entre in visitesParUtilisateur:
43         if entre[0] == utilisateur:
44             if entre[2] > max_visites:
45                 max_visites = entre[2]
46                 url_max = entre[1]
47
```



```

47
48 # Afficher les utilisateurs ainsi que la l'URL qui ont le plus visités
49 print(f"Utilisateur : {utilisateur}")
50 print(f"URL la plus visitée : {url_max} avec {max_visites} visites\n")
51

```

Le programme présente, pour chaque utilisateur, l'URL la plus visitée avec son nombre de visites, être stocké dans une variable appropriée.

Etape 6 :

Etape 7 :

```

generate_sql_insert.py
Fichiers logs disponibles :
1. log_proxy_2024-11-23.txt
2. log_proxy_2024-11-24.txt
3. log_proxy_2024-11-25.txt
Entrez le numéro du fichier à analyser : 1
Fichier sélectionné : log_proxy_2024-11-23.txt
Script SQL généré : insert_log_proxy_2024-11-23.sql

```

```

generate_sql_insert.py
Fichiers logs disponibles :
1. log_proxy_2024-11-23.txt
2. log_proxy_2024-11-24.txt
3. log_proxy_2024-11-25.txt
Entrez le numéro du fichier à analyser : 2
Fichier sélectionné : log_proxy_2024-11-24.txt
Script SQL généré : insert_log_proxy_2024-11-24.sql

```

```

generate_sql_insert.py
Fichiers logs disponibles :
1. log_proxy_2024-11-23.txt
2. log_proxy_2024-11-24.txt
3. log_proxy_2024-11-25.txt
Entrez le numéro du fichier à analyser : 3
Fichier sélectionné : log_proxy_2024-11-25.txt
Script SQL généré : insert_log_proxy_2024-11-25.sql

```

 insert_log_proxy_2024-11-23.sql	08/01/2025 12:12	Fichier source SQL	53 Ko
 insert_log_proxy_2024-11-24.sql	08/01/2025 12:01	Fichier source SQL	303 Ko
 insert_log_proxy_2024-11-25.sql	08/01/2025 12:12	Fichier source SQL	304 Ko
 log_proxy_2024-11-23.txt	08/01/2025 09:11	Document texte	20 Ko
 log_proxy_2024-11-24.txt	03/01/2025 09:17	Document texte	112 Ko
 log_proxy_2024-11-25.txt	03/01/2025 09:17	Document texte	111 Ko

Etape 8 :

Nous procédons à l'implémentation des 3 fichiers sql dans la base de données :

Afficher la zone SQL

✓ # 1 ligne affectée.

Téléversement SQL (0)

✓ insert_log_proxy_2024-11-23.sql Succès

Afficher la zone SQL

✓ # 1 ligne affectée.

Téléversement SQL (0)

✓ insert_log_proxy_2024-11-24.sql Succès

Afficher la zone SQL

✓ # 1 ligne affectée.

Téléversement SQL (0)

✓ insert_log_proxy_2024-11-25.sql Succès

Console de requêtes SQL

✓ Affichage des lignes 0 - 24 (total de 116, traitement en 0.0861 seconde(s).) [adresse_ip_employe: 192.168.2.1... - 192.168.2.100...]

SELECT adresse_ip_employe, url_consultee, COUNT(*) as visit_count FROM journaux_acces GROUP BY adresse_ip_employe , url_consultee ORDER BY adresse_ip_employe, visit_count DESC;

☐ Profilage [Éditer en ligne] [Éditer] [Expliquer SQL] [Créer le code source PHP] [Actualiser]

1 > >>

☐ Tout afficher

Nombre de lignes : 25

Filtrer les lignes: Chercher dans cette table

Options supplémentaires

adresse_ip_employe	url_consultee	visit_count
192.168.2.1	https://www.nytimes.com	285
192.168.2.1	https://www.office.com	261
192.168.2.1	https://www.google.fr	255
192.168.2.1	https://www.disneyplus.com	255

Test de la requête SQL réaliser à l'étape 4. Elle fonctionne correctement.

Etape 9 :

Ligne de log pour l'exemple :

00:10:58 192.168.2.100 PUT 200 https://www.netflix.com 231+2460 OK

- 00:10:58 : La requête a été émise à 00h10 et 58 secondes.
- 192.168.2.100 : L'utilisateur à l'adresse IP 192.168.2.100 a envoyé la requête.
- PUT : La méthode HTTP utilisée est PUT.
- 200 : Le serveur a répondu avec un code 200 (succès).
- https://www.netflix.com : La requête ciblait l'URL Netflix.
- 231+2460 : La taille de la requête était de 231 octets et celle de la réponse de 2460 octets.
- OK : Statut indiquant que tout s'est déroulé correctement.